

第二次作业

20337025 崔璨明

1. 实验目的

1. 熟悉并掌握主成分分析的基本原理
2. 学会应用主成分分析实现数据降维，并应用到图像压缩

2. 实验要求

1. 提交实验报告，要求有适当步骤说明和结果分析
2. 将代码和结果打包提交
3. 不能直接调用现有的库函数提供的 PCA 接口

3. 实验内容

1. 按照主成分分析的原理实现 PCA 函数接口
2. 利用实现的 PCA 函数对图像数据进行压缩和重建
3. 利用实现的 PCA 函数对高维数据进行低维可视化

4. 实验过程

4-1 实现 PCA 函数接口

4-1-1 实验原理

PCA的核心思想是寻找数据中的主成分，即在数据中最能够代表数据变化的方向。在图像中，这些主成分可以被视为代表了图像中的重要信息。因此，要将图像进行压缩，可以先将图像转化为矩阵形式，其中每个像素点对应一个矩阵元素。然后对矩阵进行PCA分析，找出主成分。这些主成分通常是矩阵中变化最大的方向。保留部分主成分，舍弃其余主成分。舍弃的主成分代表了图像中的噪音或次要信息。保留的主成分则代表了图像中最重要的信息。最后使用保留的主成分重构原始图像。这可以通过使用保留的主成分来逆转PCA过程来实现。

由于PCA基于保留主要信息的原则，因此在压缩图像时可以实现较高的压缩比例，同时保留大部分的图像信息。

4-1-2 具体实现与代码展示

实现PCA函数的步骤如下：

1. 加载图像，并将其转换为 `numpy` 数组。
2. 将RGB图像转换为灰度图像，这将简化我们的计算。
3. 将图像数据的均值中心化，即对每个像素减去均值。
4. 计算数据的协方差矩阵。
5. 计算协方差矩阵的特征值和特征向量。
6. 将特征向量按照特征值从大到小排序。
7. 选择要保留的特征向量数量，并将其转换为投影矩阵。
8. 将图像数据乘以投影矩阵，即将数据投影到低维空间中。
9. 将投影后的数据恢复到原始空间中，即将数据乘以投影矩阵的转置。
10. 将恢复的数据加上均值，以得到最终的重建图像。

具体代码实现如下，参数 `x` 为转为数组的图像，参数 `num_components` 为指定保留的主成分数量：

```
def pca(X, num_components):
    # 计算均值并中心化数据
    X_mean = np.mean(X, axis=0)
    X_centered = X - X_mean
    # 计算协方差矩阵
    cov_matrix = np.cov(X_centered.T)
    # 对协方差矩阵进行SVD分解
    U, S, V = linalg.svd(cov_matrix)
    # 获取前num_components个主成分
    components = V.T[:, :num_components]
    # 将数据投影到主成分空间中
    projected = X_centered.dot(components)
    # 将数据重建回原始空间中
    reconstructed = projected.dot(components.T) + X_mean
    return projected, reconstructed, components
```

4-2 PCA 的基本应用

Eigen Face 数据集中的灰度人脸数据进行压缩和重建

首先加载数据，使用 `loadmat` 函数从文件中读取面部数据集，存储在`X`变量中，并保存前100张原图像用于对比。接着调用`pca`函数对数据集`X`进行PCA分析，得到投影数据、重建数据和主成分。并将前49个主成分保存。最后对于不同的主成分数，前100张压缩和重建后的图像进行展示。

```
def task1():
    # 加载数据
    data = loadmat('data/faces.mat')
    X = data['X']
    #保存前100张原图像
    for i in range(1,101):
        plt.subplot(10,10,i)
        plt.imshow(X[i-1].reshape(32, 32).T, cmap='gray')
        plt.axis('off')
    plt.savefig('results/PCA/origin_faces.jpg')
    plt.close()
    # PCA分析
    projected, reconstructed, components = pca(X, 150)
    # 展示前49个主成分
    for i in range(1,50):
        plt.subplot(7,7,i)
        plt.imshow(components[:, i-1].reshape(32, 32).T, cmap='gray')
        plt.axis('off')
    plt.savefig('results/PCA/eigen_faces.jpg')
    plt.close()
    # 压缩和重建
    for num_components in [10, 50, 100, 150]:
        projected, reconstructed, _ = pca(X, num_components)
        plt.figure()
        print(f"\nnum_components: {num_components}")
        for i in trange(1,101):
            plt.subplot(10,10,i)
            plt.imshow(reconstructed[i-1].reshape(32, 32).T, cmap='gray')
```

```
plt.axis('off')
plt.savefig(f'results/PCA/recovered_faces_top_{num_components}.jpg')
plt.close()
```

对 lena.png 彩色 RGB 图进行压缩和重建

函数执行过程如下代码注释所述，处理彩色图片需要通过 `img_np` 的切片操作，将图像的红色通道、绿色通道和蓝色通道分别存储在 `img_r`、`img_g` 和 `img_b` 变量中，然后使用 `pca` 函数对 `img_r`、`img_g` 和 `img_b` 分别进行PCA分析和压缩重建，最后将压缩重建后的红色通道、绿色通道和蓝色通道组合为一个彩色图像即可。

```
def task2():
    img=Image.open('data/lena.jpg')
    img_np = np.array(img)
    img_r = img_np[:, :, 0]
    img_g = img_np[:, :, 1]
    img_b = img_np[:, :, 2]
    # PCA分析并展示原始图像
    fig, axs = plt.subplots(1, 5, figsize=(15, 5))
    axs[0].imshow(img_np,)
    axs[0].axis('off')
    axs[0].set_title('Original')
    for i, num_components in enumerate([10, 50, 100, 150]):
        # 压缩和重建
        _,img_r_recover,_=pca(img_r, num_components)
        _,img_g_recover,_=pca(img_g, num_components)
        _,img_b_recover,_=pca(img_b, num_components)
        recover_color_img = np.dstack((img_r_recover, img_g_recover,
img_b_recover))
        recover_color_img = (recover_color_img - np.min(recover_color_img)) /
(np.max(recover_color_img) - np.min(recover_color_img))
        # 展示重建图像
        axs[i+1].imshow(recover_color_img)
        axs[i+1].axis('off')
        axs[i+1].set_title(f'Top {num_components}')
        # 保存重建图像
        plt.imsave(f'results/PCA/recovered_lena_top_{num_components}.jpg',
recover_color_img)
        print(f"Saved recoverstructed image with top {num_components}
components.")
    plt.savefig('results/PCA/recovered_lena.jpg')
```

5. 结果分析与对比

利用实现的 PCA 函数，对 Eigen Face 数据集中的灰度人脸数据进行压缩和重建，前 49 个主成分展示如下：



用实现的PCA函数对这些人脸数据降维到不同维度(10, 50, 100, 150)进行压缩，结果对比如下（完整图像见 `result` 文件夹，这里展示前10个的对比）：

原图像：



降维 10：



降维 50：



降维 100：



降维 150：



可以见，降维的维度越高，人脸图像重建后的效果越好。

用实现的PCA函数对 `lena.png` 彩色 RGB 图进行压缩和重建，将其降维到不同维度(10, 50, 100, 150)进行压缩，结果对比如下，最左边为压缩前的图像，接着的四个图像分别是降维到不同维度(10, 50, 100, 150)再重建的图像：



可以看到，降维的维度越高，重建后的效果越好。

6. 感想与总结

在本次实验中，我学习了PCA的原理和实现方法，掌握了利用python手动实现PCA函数接口，同时也了解了降维和可视化等相关概念和技术。通过实践，我加深了对这些课上所学的理论知识的理解和应用，为以后的学习和工作打下了坚实的基础。